

Concurrent Task Dispatcher Design Report

Name: Elian Pena

Project: Concurrent Task Dispatcher in Rust

Course: CSCI-3334-01-Spring 2026

This project is a Rust program that runs tasks with 4 workers. The program makes tasks, puts them in a queue, and then the workers take the tasks and run them. Each task must wait in the queue before a worker can pick it up. After the task is done, the program saves the results and uses them for the final numbers. I also compared two ways of choosing tasks. FIFO runs tasks in the order they were added, and optimized runs the shorter tasks first. I used this to see how each one changed the wait time, total run time, and worker usage.

Architecture

The program has four main parts:

Task maker → Shared queue → Worker pool → Metrics

My program works like a simple line of steps. A task is made first, then it goes into the queue, then a worker takes it, and then the final numbers get updated. I used this setup because it makes the project easier to follow. The task maker creates 500 tasks. Each task has a number, a type, a run time, and the time it was made. The task type is either CPU or IO. I used CPU and IO tasks so the program could have different types of work. The queue is where tasks wait before they run. The workers do not get the tasks right away; they must take them from the queue. This is also where the scheduling matters. FIFO keeps the tasks in the same order, and optimized puts the shorter tasks first.

The worker pool has 4 workers. I used 4 workers, so the program has a fixed group of threads instead of making a new thread for every task. Each worker checks the queue, takes a task, runs it, and updates the results. If the queue is empty, then the worker stops. The metrics keep track of what happened after the tasks finish. They count the completed tasks, CPU tasks, IO tasks, wait time, total time, and worker usage. This helped me compare FIFO and optimized.

Task Model

Each task in my program has a number, a type, a run time, and the time it was made, so the program can tell tasks apart and keep track of them. The type is either CPU or IO, and the run time tells the worker how long the task should run. The time it was made helps the program find out how long the task waited before a worker started it.

I used CPU and IO tasks so the work would not all be the same. The balanced workload has a mix of both, while the stressed workload has more CPU tasks and different run times, which makes the scheduler work harder and gives better results to compare.

This task information is also used for the final numbers, like wait time and turnaround time. Wait time means how long the task waited before running, and turnaround time means how long it took from being made to being finished.

Synchronization Strategy

The queue is shared by all 4 workers, so I had to make sure they do not mess it up when using it at the same time. I used `Arc<Mutex<VecDeque<Task>>>` for the queue because Arc lets the workers share it and Mutex makes only one worker use it at a time.

This matters because two workers should not be able to grab the same task. When a worker needs a task, it locks the queue, takes one task from the front, and then let's go of the lock, so the task can run without blocking the whole queue.

The final numbers are shared too because every worker updates them after finishing a task, so I used a Mutex there also. One downside is that workers might wait a little for the lock, but I only lock the queue when taking a task and only lock the numbers when updating them.

I did not use channels in my final design because all the workers needed to share one queue and update the same final numbers. I used shared state with Arc and Mutex instead, so the workers could safely use the same queue and metrics.

Scheduling Policy

The program has two ways to pick tasks, FIFO and optimized. FIFO takes tasks in the same order they were added to the queue, so it is easy to understand. The good part is that it is fair by order, but the bad part is that a short task might have to wait behind a long task.

Optimized puts the shorter tasks first, so small tasks can finish faster. I used this to see if it would lower the wait time. The downside is that longer tasks might have to wait more because the shorter tasks go first.

I used both modes because they are easy to compare. FIFO shows the basic way, and optimized shows what happens when the program tries to finish shorter tasks sooner also the optimized policy helped shorter tasks finish sooner because small tasks were moved earlier, but it also made the design less fair for longer tasks because they could be pushed back.

Metrics

The program prints the final numbers after all the tasks are done. It shows total tasks completed, CPU tasks completed, IO tasks completed, makespan, average wait time, average turnaround time, max wait time, and worker usage.

Makespan means how long the whole program took to finish. Wait time means how long a task waited before a worker started it. Turnaround time means how long it took from when the task was made to when it finished.

I added worker usage because it shows how busy the workers were during the run. This helps compare FIFO and optimized because I can see not only how long the program took, but also how much the workers were being used.

Experiments

I ran two main experiments. The first one was FIFO with the balanced workload, and the second one was optimized with the stressed workload. Both runs used 500 tasks and 4 workers.

For FIFO balanced, the tasks were handled in the same order they were added. This showed how the basic scheduler worked with a normal mix of CPU and IO tasks.

For optimized stressed, the shorter tasks were placed first, and the workload had more CPU tasks with different run times. This showed how the scheduler worked with a harder workload.

Results:

FIFO balanced:

Total tasks completed: 500

CPU tasks completed: 250

IO tasks completed: 250

Makespan: 4.340677327s

Average wait time: 2.148690504s

Average turnaround time: 2.178931801s

Max wait time: 3.756921446s

Worker utilization: 86.39%

Optimized stressed:

Total tasks completed: 500

CPU tasks completed: 400

IO tasks completed: 100

Makespan: 5.447982628s

Average wait time: 2.408465357s

Average turnaround time: 2.447611959s

Max wait time: 4.8532623s

Worker utilization: 89.48%

The optimized stressed run used the workers a little more, but it also took longer and had higher wait times. I think this happened because the stressed workload had more CPU tasks, so there was more heavy work for the workers to finish. The FIFO balanced run finished faster because it had an even mix of CPU and IO tasks. This showed me that putting shorter tasks first can help, but the type of workload still matters a lot.

Bug or Problem I Hit

One problem I had was making sure the workers stopped the right way. Since each worker runs in a loop, I had to make sure they did not keep running after all the tasks were done. I fixed this by checking the queue each time. If `pop_front()` gave back `None`, the worker stopped.

Another problem was the shared queue. Since all 4 workers use the same queue, I had to make sure two workers did not take the same task. I fixed this by using a Mutex, so only one worker could use the queue at once.

I also had a Rust warning early on because `time_needed` and `created_at` were not being used yet. Once I used those for running the task and tracking time, the warning went away.

Trade-Offs and Conclusion

One trade-off was using one shared queue because it made the program easier to follow, but sometimes a worker might have to wait if another worker is already using the queue. Another trade-off was FIFO compared to optimized, FIFO is simple because it keeps the same order, but a short task can get stuck behind a long task. Optimized helps shorter tasks finish faster, but longer tasks can end up waiting more, so it can still be unfair sometimes if there are a lot of short tasks. Overall, this project helped me understand how a task dispatcher works in Rust, a task is made, put into a queue, picked up by a worker, run, and then added to the final results. It also showed me that the way tasks are picked can change wait time, total run time, and worker usage.

